

## **g<sup>2</sup>(τ) Emitter Classification Pipeline**

Code accompanying the paper “*Physics-Based versus Data-Driven Classification of Single-Photon Quantum Emitters from Sparse Autocorrelation Data.*” Benchmarks three classification approaches—Levenberg–Marquardt fitting, sequential Bayesian inference, and a feedforward neural network—for distinguishing single-photon emitters ( $N=1$ ) from multi-emitter sources ( $N \geq 2$ ) from Hanbury Brown–Twiss  $g^2(\tau)$  autocorrelation data, with particular attention to performance under sparse (short-integration-time) conditions.

### **What's in this repository**

Only the **real experimental data** is included here. Simulated training/test data is not, for space reasons—anyone wishing to reproduce or extend the analysis needs to generate it themselves from the provided real data, using the three Step\_ scripts below. This is by design: it also means anyone re-running the pipeline is using the exact same generation code we did, not a static snapshot of one particular run.

- |— Experimental\_Data\_Raw/                      <- provided: real HBT measurements
- |— Step\_00\_Correct\_and\_relabel\_experimental\_data.py
- |— Step\_01\_Extract\_Params\_from\_experimental\_data.py
- |— Step\_02\_Generate\_g2\_traces\_with\_exp\_params.py
- |— LM\_analysis\_Simulation\_data\_iterspecific\_smoothed.py
- |— Bayesian\_analysis\_simulation\_data\_iterspecific.py
- |— Encoder\_analysis\_simulation\_data\_smoothed.py
- |— README.md

Running the pipeline (see below) will additionally create:

- |— Experimental\_Data/              <- created by Step\_00
- |— experimental\_calibration.csv, calibration\_summary.csv   <- created by Step\_01
- |— Simulation\_Results/            <- created by Step\_02, read by all three algorithms

### **Requirements**

Python 3.9+ with:

numpy  
pandas  
scipy

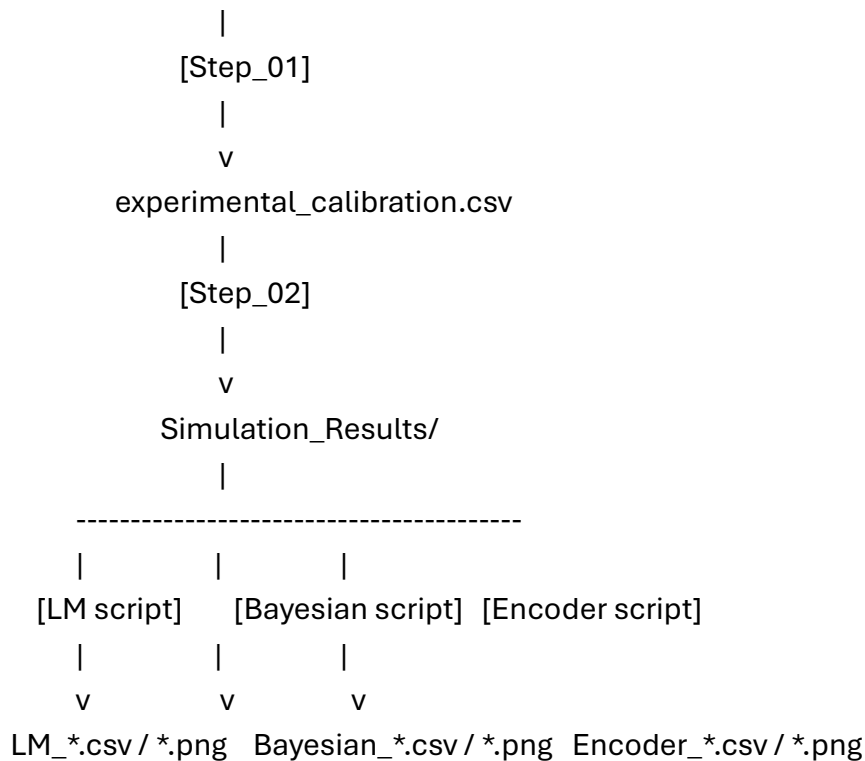
matplotlib  
seaborn  
scikit-learn

No GPU or deep learning framework is required—the neural-network classifier uses scikit-learn's MLPClassifier, not PyTorch/TensorFlow.

### Pipeline overview

The six scripts run in a strict sequence: each one reads the previous step's output. There is no orchestration script — run them one at a time from the command line (python Step\_00\_...py, etc.), in the same working directory, in this order:

Experimental\_Data\_Raw/ --[Step\_00]--> Experimental\_Data/



---

### Experimental\_Data\_Raw/ — real data format (provided)

Each file is one HBT measurement, named <anything>rep\_<ID>\_n<N>.csv or <anything>rep\_<ID>\_n<N>\_raw.csv (the \_raw suffix and any prefix before rep\_ are both optional — Step\_00 matches on the rep\_<ID>\_n<N> part). <N> is the emitter number as originally labeled during acquisition; Step\_00 re-derives and may correct this (see below).

File structure (CSV, no index column):

- **Row 1:** APD1, followed by one count-rate value (Hz) per iteration column — the single-channel count rate on detector 1 at each integration time.
- **Row 2:** APD2, same for detector 2.
- **Row 3** (header row): tau, iter\_1, iter\_2, ..., iter\_9.
- **Remaining rows:** tau (ns) plus one column of **raw, unnormalized** coincidence counts per iteration — each iter\_k column is the *cumulative* histogram after that many integration steps (i.e. iter\_9 contains all counts accumulated up to and including the longest integration time, not just that step's counts in isolation).

The integration time (seconds) corresponding to each iter\_k column is fixed by acquisition schedule, not stored in the file itself — it's hardcoded in Step\_00 and Step\_01 as [1, 2, 5, 10, 20, 50, 100, 200, 500] for iter\_1...iter\_9. **If your acquisition used a different schedule, update ITERATION\_INTEGRATION\_TIMES\_S in both Step\_00 and Step\_01 before running anything.**

Optionally, a file ending in \_norm.csv (otherwise identically named) can accompany a \_raw.csv file — the instrument's own normalized  $g^2(\tau)$  output for the same measurement. If present, Step\_00 uses it to derive an exact baseline; if absent, Step\_00 falls back to computing the baseline from the APD rates directly (accurate to within 1-3%, validated against files where both are available).

---

### Step\_00 — Step\_00\_Correct\_and\_relabel\_experimental\_data.py

**Reads:** Experimental\_Data\_Raw/ **Writes:** Experimental\_Data/

Corrects a small, fixed timing offset between the two detector channels (typically a few nanoseconds to tens of nanoseconds, detected per file and re-centered so the antibunching dip sits at  $\tau=0$ ), and independently re-derives each file's true emitter number from a classical amplitude threshold on the corrected  $g^2(0)$  — since a file's original N label (from the filename) may not survive timing correction. Files whose corrected  $g^2(0)$  falls too close to a classification threshold are excluded as genuinely ambiguous rather than forced into a class.

For each surviving file, writes two outputs to Experimental\_Data/:

- rep\_<ID>\_n<N>.csv — same tau/iter\_ column format as the input, but corrected and properly normalized to  $g^2(\tau)$ .
- rep\_<ID>\_n<N>\_rawcounts.csv — a companion file with the same corrected data as raw (unnormalized) counts, used downstream for proper Poisson-weighted fitting.

Prints a full audit log (detected shift, both  $g^2(0)$  estimation methods, and the reason for every exclusion) to the console — worth reading through once to sanity-check your own data before proceeding.

**Run:** python Step\_00\_Correct\_and\_relabel\_experimental\_data.py

---

### **Step\_01 — Step\_01\_Extract\_Params\_from\_experimental\_data.py**

**Reads:** Experimental\_Data/ (Step\_00's output) **Writes:** experimental\_calibration.csv, calibration\_summary.csv

Extracts, from each corrected real file, the background fraction  $f_{bg}$  (and bunching pedestal  $A_2$ ) needed to calibrate realistic noise levels in the synthetic data generated by Step\_02.  $f_{bg} = B/(S+B)$ , recovered by inverting the emitter-dilution relation  $g^2(0) = 1 - \rho^2/N$  for the file's known  $N$  and measured  $g^2(0)$  — not measured directly.

Results are pooled by emitter number:  $N=1$  on its own, and  $N=2$  through 5 pooled together (individually there are usually too few real multi-emitter files to characterize each  $N$  separately). If even the pooled  $N \geq 2$  group is too small to be confident,  $N=1$  data is blended in as a fallback — this and every other pooling decision is printed to the console.

**Run:** python Step\_01\_Extract\_Params\_from\_experimental\_data.py

---

### **Step\_02 — Step\_02\_Generate\_g2\_traces\_with\_exp\_params.py**

**Reads:** experimental\_calibration.csv (Step\_01's output) **Writes:** Simulation\_Results/

Generates the large synthetic dataset the three classifiers are actually trained and evaluated on — a Gillespie Monte Carlo simulation of a 3-level emitter, with the background fraction for each simulated file bootstrapped from the real calibration pool (rather than an idealized constant), so simulated noise statistics match what was actually measured.

Key parameters, all set near the top of the if `__name__ == "__main__":`-equivalent section of the script (search for TWEAKABLE GENERATION PARAMETERS):

| Parameter    | Meaning                                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| repetitions  | Number of simulated files to generate (5000 in the paper)                                                |
| max_emitters | Emitter number is drawn uniformly from 1 to this value (5 in the paper — gives ~20% Single / ~80% Multi) |

| Parameter     | Meaning                                                                                                                                                    |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| iterations    | Number of accumulated integration-time snapshots per file (300 in the paper)                                                                               |
| F_BG_OVERRIDE | Set to a number instead of None to force every file to the same background fraction, bypassing the real-data bootstrap (useful for controlled experiments) |

**This step is the most expensive one to run** — expect a runtime of several hours for 5000 repetitions at 300 iterations each, depending on hardware. Consider reducing repetitions for a first test run.

Outputs, one pair per repetition: `g2_raw_rep_<ID>_n<N>.csv` (thinned raw counts) and `g2_norm_rep_<ID>_n<N>.csv` (normalized  $g^2(\tau)$ ), plus a single `training_labels.csv` recording every repetition's ground-truth emitter number, injected background fraction, and a self-consistency check (background fraction and  $g^2(0)$  re-extracted from the simulation's own output, for comparison against what was injected).

**Run:** `python Step_02_Generate_g2_traces_with_exp_params.py`

---

### The three classifiers

All three read from `Simulation_Results/` (Step\_02's output) and share:

- The same binary task:  $N=1$  ("Single") vs.  $N \geq 2$  ("Multi").
- The same 5-fold stratified cross-validation (`n_folds=5`, `random_state=42`) — every file is used as a held-out test file exactly once, by a model/threshold that never saw it during calibration or training.
- The same set of 30 evaluated integration-time points (`iteration_stride=10`, i.e. iterations 10, 20, ..., up to iterations set in Step\_02).
- The same three output types: a metrics-vs-iteration table/plot (Recall, Precision, Leakage — see the paper's Methods section for definitions), a confusion matrix at the final iteration, and a per-file correctness heatmap across all iterations — so all three methods' outputs are directly comparable.

No settings need to change between scripts for a first run; each script's `SETTINGS` dictionary near the top can be edited if you want to point at a different `Simulation_Results` folder, change the number of folds, or adjust the smoothing strength (`ema_alpha`, LM and Encoder only).

| Script                                               | Method                                                                   | Key outputs<br>(Simulation_Results/)                                                                                                                  |
|------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| LM_analysis_Simulation_data_iterspecific_smoothed.py | Levenberg–Marquardt, per-iteration threshold calibration + smoothing     | LM_metrics_vs_iteration.csv/.png, LM_predictions_long.csv, LM_confusion_matrix_last_iteration.png, LM_correctness_heatmap.png                         |
| Bayesian_analysis_simulation_data_iterspecific.py    | Sequential Bayesian inference, per-chain-position likelihood calibration | Bayesian_metrics_vs_iteration.csv/.png, Bayesian_predictions_long.csv, Bayesian_confusion_matrix_last_iteration.png, Bayesian_correctness_heatmap.png |
| Encoder_analysis_simulation_data_smoothed.py         | Feedforward neural network (scikit-learn MLPClassifier) + smoothing      | Encoder_metrics_vs_iteration.csv/.png, Encoder_predictions_long.csv, Encoder_confusion_matrix_last_iteration.png, Encoder_correctness_heatmap.png     |

Each can be run independently and in any order, once Simulation\_Results/ exists:

```
python LM_analysis_Simulation_data_iterspecific_smoothed.py
```

```
python Bayesian_analysis_simulation_data_iterspecific.py
```

```
python Encoder_analysis_simulation_data_smoothed.py
```

**Runtime note:** the LM script recalibrates its decision threshold separately at every evaluated iteration, which costs substantially more compute than a single fixed-threshold calibration — expect it to be the slowest of the three on a full 5000-file run.

---

## Reproducibility notes

- All three classifiers use `random_state=42` for their cross-validation split; results should be exactly reproducible given the same `Simulation_Results/` data.
- `Step_02`'s Monte Carlo generation is *not* seeded, so re-running it produces a different (though statistically similar) synthetic dataset each time. If you need a byte-identical dataset across runs, add an explicit `np.random.seed(...)` near the top of `Step_02` before generation begins.
- Every script prints a full log of what it did (files excluded, folds calibrated, parameters used) to the console — worth capturing (`python Step_00_....py | tee step00_log.txt`) alongside your results for later reference.